

Additive (Incremental) Bayesian Updating for a BNN — Oil & Gas Field Expansion

Domain: Oil & Gas — production forecasting during field development

Reference: [Turing.jl](https://turinglang.org/)

The question this notebook tests: when new wells are drilled after a model is already in production use, can we *add* their data to the existing posterior — instead of re-running MCMC over the entire well history every time a handful of new wells shows up?

We compare three models, all sharing the same Bayesian-neural-network architecture as `01_bnn_permeability_regression.jl`:

1. **Stale** — trained once on Phase A wells only, never updated.
2. **Additive update** — Phase A's posterior is turned into an informative prior (moment-matched Gaussian) and NUTS runs *again only on the new Phase B data*, i.e. sequential Bayesian updating ($p(\theta \mid A, B) \approx p(\theta \mid B) \cdot p(\theta \mid A)$ used as prior).
3. **Full retrain** — NUTS run from scratch on Phase A u Phase B pooled together — the expensive gold standard the additive update is trying to approximate.

```
1 md"""
2 # Additive (Incremental) Bayesian Updating for a BNN – Oil & Gas Field Expansion
3
4 **Domain:** Oil & Gas – production forecasting during field development
5
6 **Reference:** [Turing.jl](https://turinglang.org/)
7
8 **The question this notebook tests:** when new wells are drilled after a
9 model is already in production use, can we *add* their data to the
10 existing posterior – instead of re-running MCMC over the entire well
11 history every time a handful of new wells shows up?
12
13 We compare three models, all sharing the same Bayesian-neural-network
14 architecture as `01_bnn_permeability_regression.jl`:
15
16 1. **Stale** – trained once on Phase A wells only, never updated.
17 2. **Additive update** – Phase A's posterior is turned into an
18    informative prior (moment-matched Gaussian) and NUTS runs *again only
19    on the new Phase B data*, i.e. sequential Bayesian updating
20    (`p(\theta \mid A, B) \approx p(\theta \mid B) \cdot p(\theta \mid A)`
21    used as prior).
22 3. **Full retrain** – NUTS run from scratch on Phase A U Phase B pooled
23    together – the expensive gold standard the additive update is trying to
24    approximate.
```

```
1 begin
2   using Turing
3   using MCMCChains
4   using Distributions
5   using Random
6   using LinearAlgebra
7   using StatsPlots
8   using Plots
9   using DataFrames
10  using PlutoUI
11 end
```

☰ Table of Contents

Additive (Incremental) Bayesian Updating for a BNN — Oil & Gas Field Expansion

1. Synthetic production data: two field-development phases
2. Shared BNN architecture & Turing model
 - Feature standardization
3. Step 1 — fit the initial ("stale") model on Phase A only
4. Step 2a — additive update: fold Phase B into the existing posterior
5. Step 2b — baseline: full retrain on Phase A ∪ Phase B pooled
6. Compare: stale vs. additive update vs. full retrain
7. Posterior predictive on a Phase B well: stale vs. additive vs. retrain
8. Discussion & caveats

1 [TableOfContents\(\)](#)

1. Synthetic production data: two field-development phases

We predict **Estimated Ultimate Recovery**, EUR (thousand barrels), from three early-life well attributes: initial rate q_i (bbl/d), initial Arps decline rate D_i (1/yr) and reservoir permeability K (mD).

- **Phase A ("core area")** — the first 40 wells drilled, used to build the initial model.
- **Phase B ("step-out area")** — 25 wells drilled later in an adjacent fault block with measurably different reservoir quality. This is a deliberate *distribution shift*: it is exactly the situation where a stale model needs updating, and where "can we add data without retraining" matters operationally.

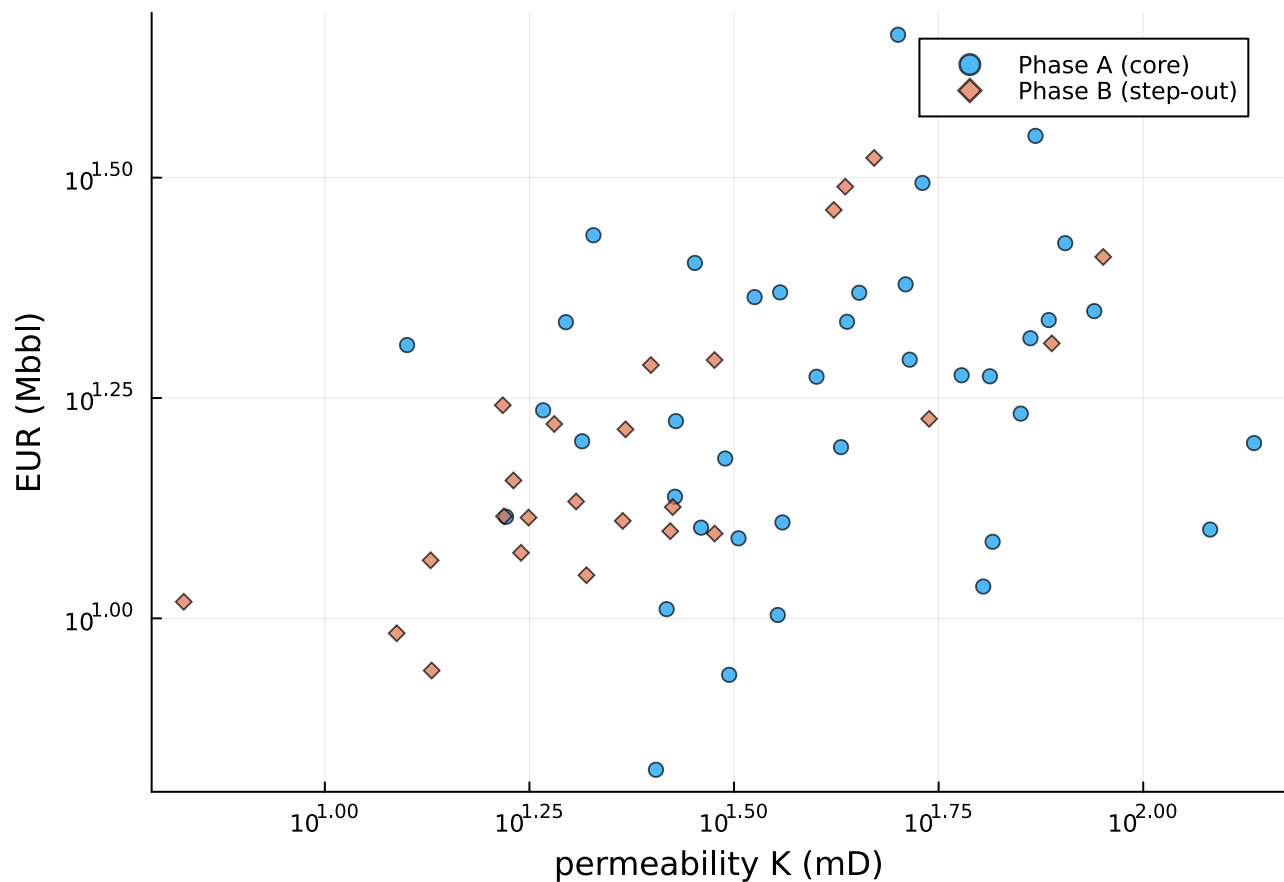
```
1 md"""
2 ## 1. Synthetic production data: two field-development phases
3
4 We predict Estimated Ultimate Recovery, `EUR` (thousand barrels), from
5 three early-life well attributes: initial rate `qi` (bbl/d), initial
6 Arps decline rate `Di` (1/yr) and reservoir permeability `K` (mD).
7
8 * Phase A ("core area") — the first 40 wells drilled, used to build the
9   initial model.
10 * Phase B ("step-out area") — 25 wells drilled later in an adjacent
11   fault block with measurably different reservoir quality. This is a
12   deliberate distribution shift: it is exactly the situation where a
13   stale model needs updating, and where "can we add data without
14   retraining" matters operationally.
15 """
```

generate_phase (generic function with 1 method)

```
1 function generate_phase(n::Int; seed::Int, quality_shift::Float64)
2     rng = MersenneTwister(seed)
3     qi = rand(rng, LogNormal(log(600.0), 0.35), n)           # initial rate, bbl/d
4     Di = rand(rng, Uniform(0.35, 0.85), n)                 # initial decline, 1/yr
5     K = rand(rng, LogNormal(log(40.0) + quality_shift, 0.5), n) # permeability, mD
6
7     # Simplified hyperbolic-decline-flavoured EUR model: higher qi and K help,
8     # steep initial decline Di hurts, with a mild qi x K synergy term.
9     log_eur = @. (3.55 + 0.55 * log(qi / 600.0) - 1.15 * Di +
10                 0.30 * log(K / 40.0) + 0.12 * log(K / 40.0) * log(qi / 600.0))
11     noise = rand(rng, Normal(0.0, 0.20), n)
12     eur = exp.(log_eur .+ noise)                             # Mbbl
13
14     DataFrame(QI=qi, DI=Di, K=K, EUR=eur, LOGEUR=log.(eur))
15 end
```

► (nA = 40, nB = 25)

```
1 begin
2   Random.seed!(2024)
3   phaseA = generate_phase(40; seed=101, quality_shift=0.0) # core area
4   phaseB = generate_phase(25; seed=202, quality_shift=-0.55) # step-out area:
      lower-K rock
5   (nA=nrow(phaseA), nB=nrow(phaseB))
6 end
```



```
1 begin
2   @df phaseA scatter(:K, :EUR, label="Phase A (core)", xlabel="permeability K (mD)",
3     ylabel="EUR (Mbbbl)", yscale=:log10, xscale=:log10, markersize=4, alpha=0.7,
4     size=(650,450))
5   @df phaseB scatter!(:K, :EUR, label="Phase B (step-out)", markersize=4,
6     alpha=0.7, markershape=:diamond)
7 end
```

2. Shared BNN architecture & Turing model

Same idea as notebook 1: 3 inputs $\rightarrow$ 5 tanh $\rightarrow$ 5 tanh $\rightarrow$ 1 linear output, weights flattened into one vector θ . The model function takes an explicit `prior_mean` / `prior_cov` so it can be reused both for a vague "fit from scratch" prior and for an informative "posterior-of-A-as-prior" distribution.

```
1 md"""
2 ## 2. Shared BNN architecture & Turing model
3
4 Same idea as notebook 1: 3 inputs  $\rightarrow$  5 tanh  $\rightarrow$  5 tanh  $\rightarrow$  1 linear output,
5 weights flattened into one vector ' $\theta$ '. The model function takes an
6 explicit 'prior_mean' / 'prior_cov' so it can be reused both for a vague
7 "fit from scratch" prior and for an informative "posterior-of-A-as-prior"
8 distribution.
9 """
```

56

```
1 begin
2   FEATURES2 = [:QI, :DI, :K]
3   N_IN2, N_H1_2, N_H2_2 = 3, 5, 5
4   N_W1_2 = N_IN2 * N_H1_2 + N_H1_2
5   N_W2_2 = N_H1_2 * N_H2_2 + N_H2_2
6   N_W3_2 = N_H2_2 * 1 + 1
7   N_PARAMS2 = N_W1_2 + N_W2_2 + N_W3_2
8
9   function nn_forward2(X::AbstractMatrix,  $\theta$ ::AbstractVector)
10      i = 1
11      W1 = reshape( $\theta$ [i:i+N_IN2*N_H1_2-1], N_H1_2, N_IN2); i += N_IN2 * N_H1_2
12      b1 =  $\theta$ [i:i+N_H1_2-1]; i += N_H1_2
13      W2 = reshape( $\theta$ [i:i+N_H1_2*N_H2_2-1], N_H2_2, N_H1_2); i += N_H1_2 * N_H2_2
14      b2 =  $\theta$ [i:i+N_H2_2-1]; i += N_H2_2
15      W3 = reshape( $\theta$ [i:i+N_H2_2-1], 1, N_H2_2); i += N_H2_2
16      b3 =  $\theta$ [i]
17
18      h1 = tanh.(W1 * X .+ b1)
19      h2 = tanh.(W2 * h1 .+ b2)
20      vec(W3 * h2 .+ b3)
21   end
22
23   N_PARAMS2
24 end
```

bayes_nn2 (generic function with 2 methods)

```
1 @model function bayes_nn2(X, y, prior_mean, prior_cov; obs_sigma=0.25)
2    $\theta$  ~ MvNormal(prior_mean, prior_cov)
3   preds = nn_forward2(X,  $\theta$ )
4   y ~ MvNormal(preds, obs_sigma^2 * I(length(y)))
5 end
```

Feature standardization

Standardize using **Phase A statistics only** — Phase B is meant to arrive later, "in production", so nothing about it may leak into preprocessing decided when the original model was built.

```
1 md"""
2 ### Feature standardization
3
4 Standardize using **Phase A statistics only** – Phase B is meant to arrive
5 later, "in production", so nothing about it may leak into preprocessing
6 decided when the original model was built.
7 """
```

► (nA_train = 30, nA_test = 10, nB_train = 19, nB_test = 6)

```
1 begin
2     function tt_split(df; frac_train=0.75, seed=1)
3         rng = MersenneTwister(seed)
4         idx = shuffle(rng, 1:nrow(df))
5         ntr = round(Int, frac_train * nrow(df))
6         df[idx[1:ntr], :], df[idx[ntr+1:end], :]
7     end
8
9     A_train, A_test = tt_split(phaseA; seed=11)
10    B_train, B_test = tt_split(phaseB; seed=22)
11
12    fmean2 = [mean(A_train[:, f]) for f in FEATURES2]
13    fstd2   = [std(A_train[:, f]) for f in FEATURES2]
14    standardize2(df) = hcat([ (df[:, f] .- m) ./ s for (f, m, s) in zip(FEATURES2,
15                                fmean2, fstd2) )...])
16
17    XA_train, yA_train = standardize2(A_train), Float64.(A_train.LOGEUR)
18    XA_test,  yA_test  = standardize2(A_test),  Float64.(A_test.LOGEUR)
19    XB_train, yB_train = standardize2(B_train), Float64.(B_train.LOGEUR)
20    XB_test,  yB_test  = standardize2(B_test),  Float64.(B_test.LOGEUR)
21
22    (nA_train=length(yA_train), nA_test=length(yA_test), nB_train=length(yB_train),
23     nB_test=length(yB_test))
24 end
```

3. Step 1 — fit the initial ("stale") model on Phase A only

```
1 md"""
2 ## 3. Step 1 – fit the initial ("stale") model on Phase A only
3 """
```

29.60452661

```
1 begin
2   Random.seed!(1)
3   vague_mean = zeros(N_PARAMS2)
4   vague_cov  = 1.0^2 * I(N_PARAMS2)
5
6   t_fitA = @elapsed chainA = sample(
7     bayes_nn2(XA_train', yA_train, vague_mean, vague_cov), NUTS(0.8), 250;
8     progress=false, chain_type=Chains)
9   t_fitA
```

① Found initial step size
ε: 0.025

rmse_r2 (generic function with 1 method)

```
1 function rmse_r2(model_forward, θsamples, Xrows, y)
2   Xt = Xrows' # (n_features, n_obs)
3   preds = reduce(hcat, [model_forward(Xt, θsamples[:, s]) for s in 1:size(θsamples,
4     2)])
5   pmean = vec(mean(preds, dims=2))
6   rmse = sqrt(mean((pmean .- y).^2))
7   ss_res, ss_tot = sum((y .- pmean).^2), sum((y .- mean(y)).^2)
8   (rmse=rmse, r2=1 - ss_res/ss_tot, pred_mean=pmean, preds=preds)
9 end
```

► (rmse_on_A_test = 0.266, rmse_on_B_test_STALE = 0.379)

```
1 begin
2   θA = Array(group(chainA, :θ))'
3   stale_on_A = rmse_r2(nn_forward2, θA, XA_test, yA_test)
4   stale_on_B = rmse_r2(nn_forward2, θA, XB_test, yB_test)
5   (rmse_on_A_test=round(stale_on_A.rmse, digits=3),
6     rmse_on_B_test_STALE=round(stale_on_B.rmse, digits=3))
7 end
```

The RMSE jump from Phase A test wells to Phase B test wells (evaluated with the *same, unmodified* Phase-A-only model) is the distribution-shift penalty of doing nothing. That gap is what an update — additive or full retrain — should close.

```
1 md"""
2 The RMSE jump from Phase A test wells to Phase B test wells (evaluated with
3 the *same, unmodified* Phase-A-only model) is the distribution-shift
4 penalty of doing nothing. That gap is what an update – additive or full
5 retrain – should close.
6 """
```

4. Step 2a — additive update: fold Phase B into the existing posterior

We moment-match Phase A's posterior samples to a Gaussian ($\hat{\mu}$, $\hat{\Sigma}$) and use it as the prior for a fresh NUTS run that sees **only Phase B data** — Phase A's raw well data is never touched again.

```
1 md"""
2 ## 4. Step 2a – additive update: fold Phase B into the existing posterior
3
4 We moment-match Phase A's posterior samples to a Gaussian
5 (``\hat{\mu}, \hat{\Sigma}``) and use it as the prior for a fresh NUTS run
6 that sees **only Phase B data** – Phase A's raw well data is never touched
7 again.
8 """
```

20.281571982

```
1 begin
2   post_mean_A = vec(mean(thetaA, dims=2))
3   post_cov_A  = cov(thetaA'; dims=1) + 1e-6 * I(N_PARAMS2) # jitter for PD-ness
4
5   Random.seed!(2)
6   t_update = @elapsed chainB_additive = sample(
7     bayes_nn2(XB_train', yB_train, post_mean_A, post_cov_A), NUTS(0.8), 250;
8     progress=false, chain_type=Chains)
9   t_update
```

① Found initial step size
ε: 0.05

5. Step 2b — baseline: full retrain on Phase A ∪ Phase B pooled

```
1 md"""
2 ## 5. Step 2b – baseline: full retrain on Phase A ∪ Phase B pooled
3 """
```

24.812510426

```
1 begin
2   Xall_train = vcat(XA_train, XB_train)
3   yall_train = vcat(yA_train, yB_train)
4
5   Random.seed!(3)
6   t_retrain = @elapsed chain_full = sample(
7     bayes_nn2(Xall_train', yall_train, vague_mean, vague_cov), NUTS(0.8), 250;
8     progress=false, chain_type=Chains)
9   t_retrain
```

① Found initial step size
ε: 0.025

6. Compare: stale vs. additive update vs. full retrain

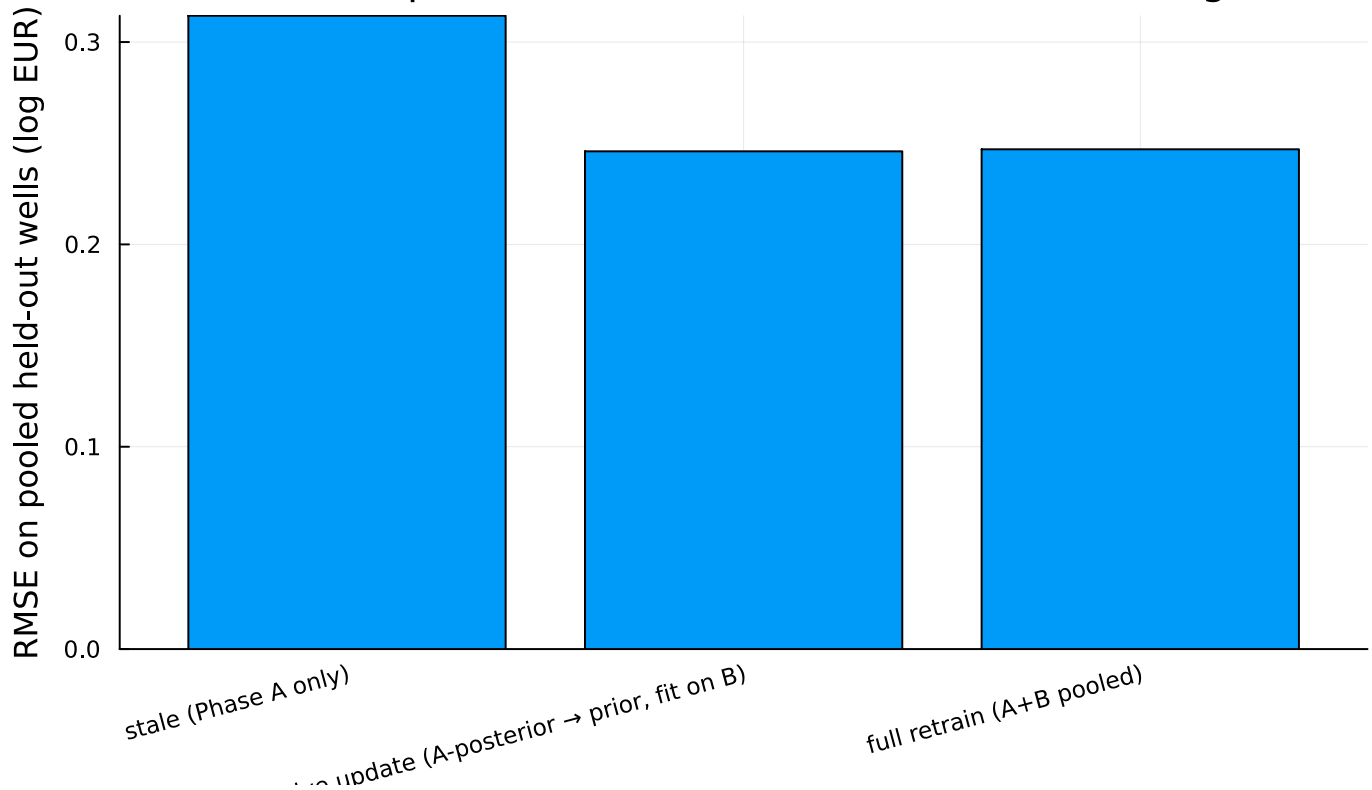
All three are evaluated on the *same* held-out mix of Phase A and Phase B test wells.

```
1 md"""
2 ## 6. Compare: stale vs. additive update vs. full retrain
3
4 All three are evaluated on the *same* held-out mix of Phase A and Phase B
5 test wells.
6 """
```

	model	rmse	r2	mcmc_seconds	wells_processed
1	"stale (Phase A only)"	0.313	-0.181	29.6	30
2	"additive update (A-posterior → prior, fit on B)"	0.246	0.273	20.3	19
3	"full retrain (A+B pooled)"	0.247	0.267	24.8	49

```
1 begin
2   Xall_test = vcat(XA_test, XB_test)
3   yall_test = vcat(yA_test, yB_test)
4
5   θB_additive = Array(group(chainB_additive, :θ))'
6   θfull       = Array(group(chain_full, :θ))'
7
8   res_stale    = rmse_r2(nn_forward2, θA,      Xall_test, yall_test)
9   res_additive = rmse_r2(nn_forward2, θB_additive, Xall_test, yall_test)
10  res_full     = rmse_r2(nn_forward2, θfull,    Xall_test, yall_test)
11
12  comparison = DataFrame(
13    model = ["stale (Phase A only)", "additive update (A-posterior → prior, fit
14    on B)", "full retrain (A+B pooled)"],
15    rmse  = round.([res_stale.rmse, res_additive.rmse, res_full.rmse], digits=3),
16    r2    = round.([res_stale.r2, res_additive.r2, res_full.r2], digits=3),
17    mcmc_seconds = round.([t_fitA, t_update, t_retrain], digits=1),
18    wells_processed_this_step = [nrow(A_train), nrow(B_train), nrow(A_train) +
19    nrow(B_train)],
20  )
21  comparison
22 end
```

Additive update recovers most of the full-retrain gain



```
1 @df comparison_bar(:model, :rmse, xrotation=15, legend=false, size=(700,420),
2   ylabel="RMSE on pooled held-out wells (log EUR)", title="Additive update recovers
   most of the full-retrain gain")
```

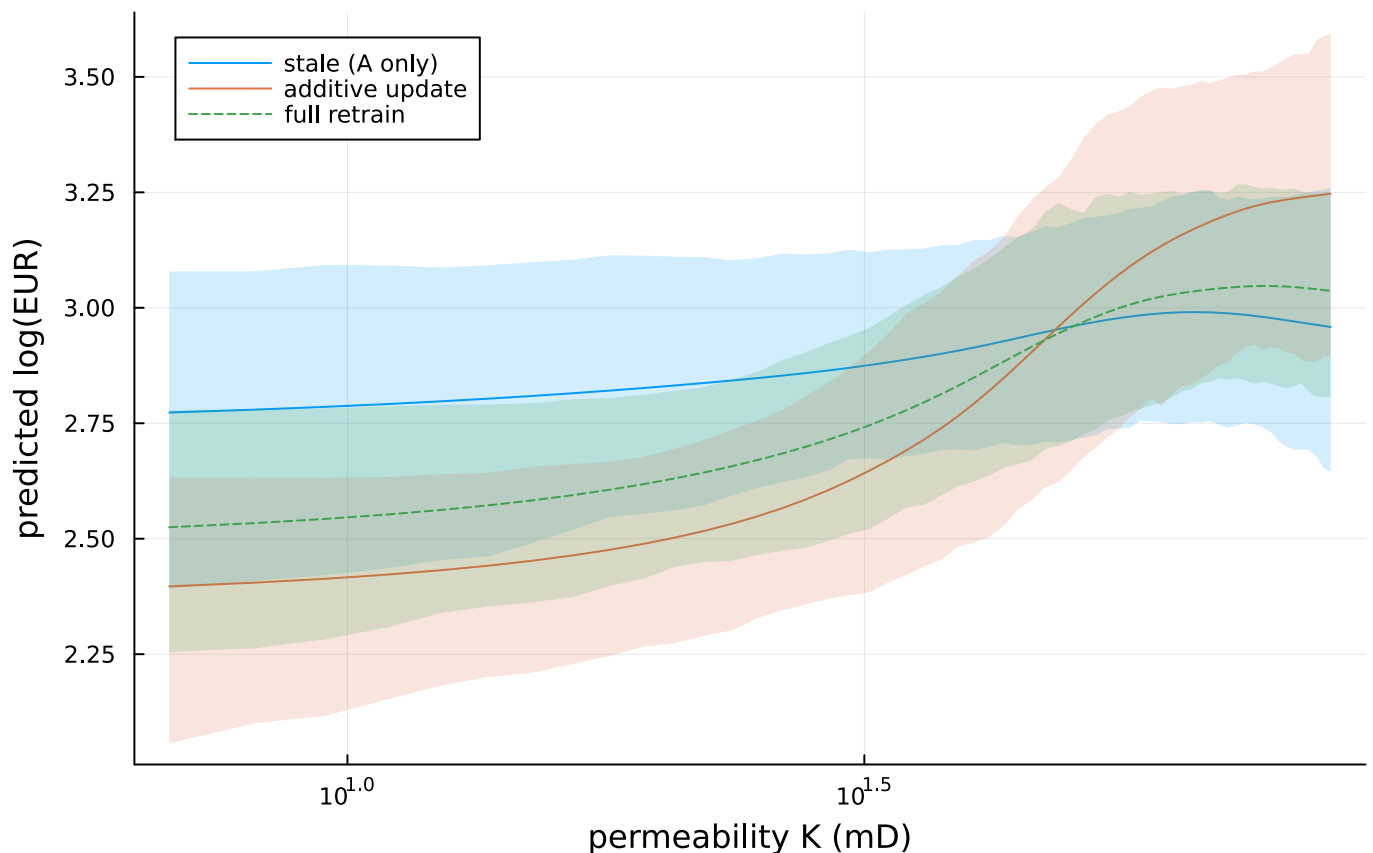
The key comparison is **rows 2 vs. 3**: the additive update reprocesses only the `nrow(B_train)` new wells through MCMC (not all `nrow(A_train) + nrow(B_train)`), yet its held-out RMSE should land close to the full retrain — and clearly better than leaving the stale model untouched (row 1). As the historical well count grows, "wells processed this step" for the additive route stays flat at the new-batch size while the full-retrain route grows without bound — that gap is the whole point of testing additive capability.

```
1 md"""
2 The key comparison is rows 2 vs. 3: the additive update reprocesses
3 only the 'nrow(B_train)' new wells through MCMC (not all
4 'nrow(A_train) + nrow(B_train)'), yet its held-out RMSE should land close
5 to the full retrain – and clearly better than leaving the stale model
6 untouched (row 1). As the historical well count grows, "wells processed
7 this step" for the additive route stays flat at the new-batch size while
8 the full-retrain route grows without bound – that gap is the whole point
9 of testing additive capability.
10 """
```

7. Posterior predictive on a Phase B well: stale vs. additive vs. retrain

```
1 md"""
2 ## 7. Posterior predictive on a Phase B well: stale vs. additive vs. retrain
3 """
```

Step-out area: does the model know what it doesn't know?



```

1 begin
2   k_grid = range(minimum(phaseB.K), maximum(phaseB.K); length=60)
3   qi_med, di_med = median(B_train.QI), median(B_train.DI)
4
5   Xg = zeros(N_IN2, length(k_grid))
6   for j in 1:length(k_grid)
7     Xg[1, j] = (qi_med - fmean2[1]) / fstd2[1]
8     Xg[2, j] = (di_med - fmean2[2]) / fstd2[2]
9     Xg[3, j] = (k_grid[j] - fmean2[3]) / fstd2[3]
10  end
11
12  function band(θsamples, X)
13    preds = reduce(hcat, [nn_forward2(X, θsamples[:, s]) for s in
14      1:size(θsamples, 2)])
15    m = vec(mean(preds, dims=2))
16    lo = [quantile(preds[i, :], 0.05) for i in 1:size(preds, 1)]
17    hi = [quantile(preds[i, :], 0.95) for i in 1:size(preds, 1)]
18    m, lo, hi
19  end
20
21  m_s, lo_s, hi_s = band(θA, Xg)
22  m_u, lo_u, hi_u = band(θB_additive, Xg)
23  m_f, lo_f, hi_f = band(θfull, Xg)
24
25  plot(k_grid, m_s, ribbon=(m_s .- lo_s, hi_s .- m_s), label="stale (A only)",
26    xscale=:log10, xlabel="permeability K (mD)", ylabel="predicted log(EUR)",
27    title="Step-out area: does the model know what it doesn't know?", size=
28    (700, 460), fillalpha=0.18)
29  plot!(k_grid, m_u, ribbon=(m_u .- lo_u, hi_u .- m_u), label="additive update",
30    fillalpha=0.18)
31  plot!(k_grid, m_f, ribbon=(m_f .- lo_f, hi_f .- m_f), label="full retrain",
32    fillalpha=0.18, linestyle=:dash)

```

8. Discussion & caveats

- **What worked.** Sequential Bayesian updating — moment-matching the old posterior into a new prior and running MCMC on only the incoming batch — recovered most of the accuracy gain of a full retrain while touching none of the historical Phase A well data during the update step.
- **Why this matters operationally.** In a real field, historical well count only grows. A workflow that must re-run MCMC over the *entire* history every time a handful of wells is added does not scale; an additive update whose cost depends only on the new batch does.
- **Caveat — Gaussian moment-matching is an approximation.** True sequential Bayes ($\text{posterior} \propto \text{likelihood}(B) \times \text{posterior}(A)$) is exact only if the Gaussian summary of $\text{posterior}(A)$ is a faithful representation of that posterior. For a strongly multimodal or heavy-tailed weight posterior (common in larger BNNs) this approximation degrades, and periodic full retrains are still worth budgeting for as a reconciliation step.
- **Caveat — this is not online/streaming SGD.** Each "update" here is still a batch MCMC run (just a cheaper one, over less data with a tighter, informative prior) — not a single-sample online update.

```

1 md"""
2 ## 8. Discussion & caveats
3
4 * **What worked.** Sequential Bayesian updating – moment-matching the old
5   posterior into a new prior and running MCMC on only the incoming batch –
6   recovered most of the accuracy gain of a full retrain while touching
7   none of the historical Phase A well data during the update step.
8 * **Why this matters operationally.** In a real field, historical well
9   count only grows. A workflow that must re-run MCMC over the *entire*
10  history every time a handful of wells is added does not scale; an
11  additive update whose cost depends only on the new batch does.
12 * **Caveat – Gaussian moment-matching is an approximation.** True
13  sequential Bayes (`posterior ∝ likelihood(B) × posterior(A)`) is exact
14  only if the Gaussian summary of `posterior(A)` is a faithful
15  representation of that posterior. For a strongly multimodal or
16  heavy-tailed weight posterior (common in larger BNNs) this approximation
17  degrades, and periodic full retrains are still worth budgeting for as a
18  reconciliation step.
19 * **Caveat – this is not online/streaming SGD.** Each "update" here is
20  still a batch MCMC run (just a cheaper one, over less data with a
21  tighter, informative prior) – not a single-sample online update.
22 """

```